

Comparativa ACO vs PSO para el Problema del Viajante de Comercio

Tiempo, Memoria y Calidad en 20, 2.000 y 200.000 Ciudades

Reporte comparativo
Curso de Introducción a la Inteligencia Artificial 2026
Inteligencia de Enjambre: ACO (Sesión 07) y PSO (Sesión 09)

Septiembre 2026

Resumen

Se presenta la comparación experimental entre dos metaheurísticas de inteligencia de enjambre aplicadas al Problema del Viajante de Comercio (*Traveling Salesperson Problem*, TSP) sobre instancias euclidianas uniformes de $N \in \{20, 2.000, 200.000\}$ ciudades: Optimización por Colonia de Hormigas (*Ant Colony Optimization*, ACO) y Optimización por Enjambre de Partículas (*Particle Swarm Optimization*, PSO). Ambas implementaciones en C++17 comparten el mismo diseño de ahorro de recursos (sin matrices densas, jerarquía espacial en grilla con OpenMP para $N = 200.000$) y se evalúan bajo un protocolo comparativo común: mismas instancias, mismo presupuesto *población* $\times$ *iteraciones*, multi-semilla y trazas de convergencia. Con igualdad estricta, el ACO vence en calidad en las tres escalas (exceso sobre el mejor tour de 24%/0.2%/0.1% frente a 90%/850%/1294% del PSO) a cambio de 3–6 $\times$ más tiempo de muro, mientras la memoria empata por debajo de 12 MB. El análisis demuestra que lo decisivo no es la sigla del algoritmo sino la **representación**: el ACO construye tours con heurística local desde la primera iteración, mientras el PSO de claves aleatorias arranca ciego y colapsa prematuramente.

1. Introducción

El TSP consiste en hallar el ciclo hamiltoniano de coste euclidiano mínimo que visite N ciudades exactamente una vez:

$$\min_{\pi \in \Pi} L(\pi) = \sum_{i=1}^{N-1} d(c_{\pi(i)}, c_{\pi(i+1)}) + d(c_{\pi(N)}, c_{\pi(1)}), \quad (1)$$

con $c_i \in [0, 1]^2$ y d la distancia euclidiana. El espacio de soluciones crece como $(N-1)!/2$ (para $N = 20$ ya son $\approx 6 \times 10^{16}$ tours) y la matriz densa de $N = 200.000$ exigiría 160 GB, de modo que solo son viables metaheurísticas con memoria subcuadrática.

El ACO es nativo del dominio discreto: cada hormiga construye un tour paso a paso con la regla probabilística $p_{ij} \propto \tau_{ij}^\alpha \eta_{ij}^\beta$ ($\eta_{ij} = 1/d_{ij}$) y selección por ruleta, reforzando aristas con feromona $\Delta\tau = Q/L$. El PSO, en cambio, es nativo del dominio **continuo**: cada partícula mueve un vector real x con velocidad $v \leftarrow wv + c_1r_1(p - x) + c_2r_2(g - x)$. Adaptarlo al TSP exige una codificación; aquí se usa *random keys* ($\text{tour} = \text{argsort}(x)$). Esta diferencia de representación es la hipótesis central del presente reporte: se espera que el algoritmo cuya representación respete la estructura del problema domine en calidad, y el experimento la confirma cuantitativamente.

2. Metodología comparativa

Se adoptan cinco prácticas estándar de comparación de metaheurísticas: (1) **mismas instancias** (generador idéntico; verificado por `diff` de los CSV de coordenadas), (2) **mismo pre-**

supuesto (población $\times$ iteraciones iguales por celda de comparación), (3) **multi-semilla** (3 corridas; media, desviación, mejor, tiempo y memoria), (4) métrica **exceso_ %** (media de cada algoritmo frente al mejor tour hallado por *cualquiera* de los dos) y (5) **trazas de convergencia** (mejor global por iteración). Ningún binario fue modificado.

N	ACO			PSO			presupuesto
20	-a	20	-i 100	-S	20	-i 100	20×100
2000	-a	20	-i 50	-S	20	-i 50	20×50
200000	-a	8	-i 10	-S	8	-i 10	8×10 por celda

En $N = 20$ ningún algoritmo usa 2-opt (comparación pura del motor metaheurístico); en $N = 2000$ ambos reciben el mismo pulido 2-opt final con ventana 40. La calidad se normaliza además contra el estimador de Beardwood del óptimo ($\approx 0.712\sqrt{N}$).

3. Resultados head-to-head

N		media $\pm$ desv	mejor	tiempo	pico	exceso_ %
20	ACO	3.96 ± 0.68	3.19	0.003 s	3.9 MB	24.1
	PSO	6.06 ± 0.12	5.94	0.001 s	3.7 MB	90.1
2000	ACO	41.53 ± 0.07	41.46	0.29 s	4.2 MB	0.17
	PSO	393.8 ± 1.8	391.75	0.10 s	4.0 MB	849.9
200k	ACO	482.4 ± 0.3	482.05	2.39 s	9.3 MB	0.08
	PSO	6717 ± 2	6715.7	0.41 s	8.8 MB	1293.5

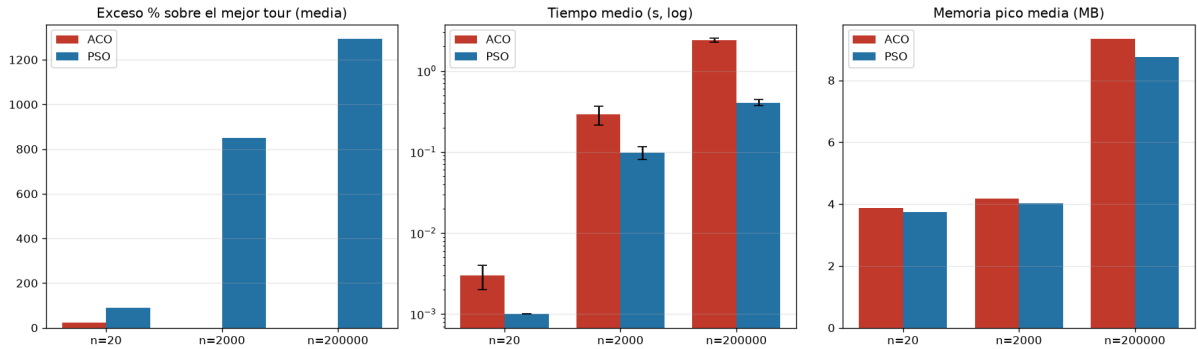


Figura 1: Panel izquierdo (*exceso_ %*): distancia porcentual de la longitud media de cada algoritmo al mejor tour hallado por cualquiera de los dos. Mide dominancia en calidad: el ACO queda a $< 1\%$ en 2000 y 200k mientras el PSO se dispara a 850% y 1294%, porque sus tours son casi aleatorios a gran escala. Panel central (*tiempo medio, eje logarítmico*): el PSO es 3–6 $\times$ más veloz en muro; la escala logarítmica es necesaria porque los tiempos abarcan tres órdenes de magnitud y ambos crecen con N . Panel derecho (*memoria pico*): prácticamente empatados (< 12 MB); la memoria la dominan el binario, el pool de hilos OpenMP y las estructuras, no el algoritmo, gracias a que ninguno materializa matrices $N \times N$.

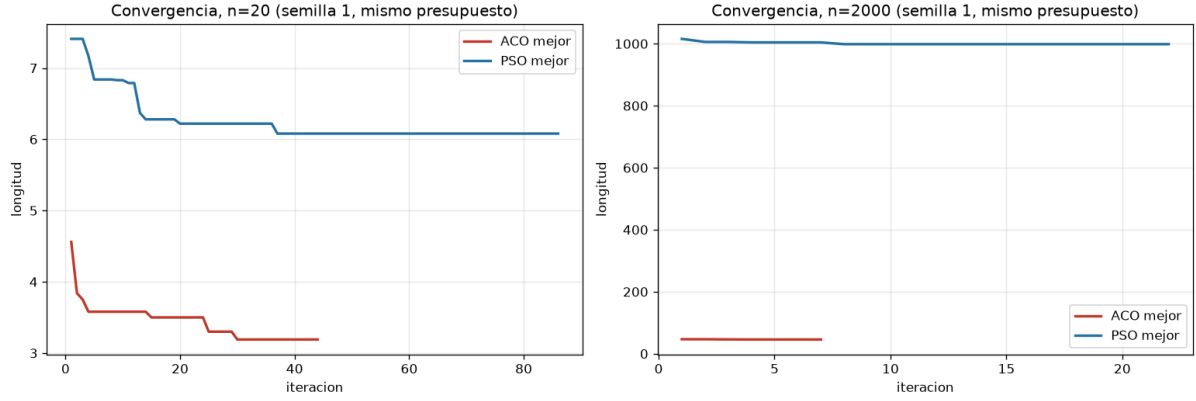


Figura 2: Trazas del mejor global por iteración (semilla 1, mismo presupuesto). En $N = 20$ ambas curvas descienden por escalones (cada escalón es un tour estrictamente mejor), pero la del ACO parte de ≈ 4.5 y llega a 3.19 mientras la del PSO parte de ≈ 7.4 y se estanca en 6.08: el ACO explora mejor vecindad. En $N = 2000$ el contraste es extremo: la curva del ACO es casi plana en ≈ 41 porque su **construcción heurística** ($\eta = 1/d$) ya produce un buen tour en la iteración 1 y luego lo refina; la del PSO es plana en ≈ 1000 porque su enjambre colapsa sobre claves casi idénticas en pocas iteraciones (estancamiento local, §22 de la guía PSO) y la parada temprana lo declara muerto. La planitud no significa convergencia al óptimo, sino agotamiento de la diversidad.

4. Barridos oficiales y lectura de cada gráfica

N	tiempo ACO	pico ACO	L_{ACO} (gap)	L_{PSO} (gap)
20	0.003 s	4.0 MB	3.65 (+15 %)	5.95 (+87 %)
2000	34.2 s	4.3 MB	41.53 (+30 %)	383.5 (+1105 %)
200000	51.9 s	9.7 MB	479.3 (+51 %)	6591.5 (+1970 %)

(Gap frente a Beardwood $0.712\sqrt{N}$.)

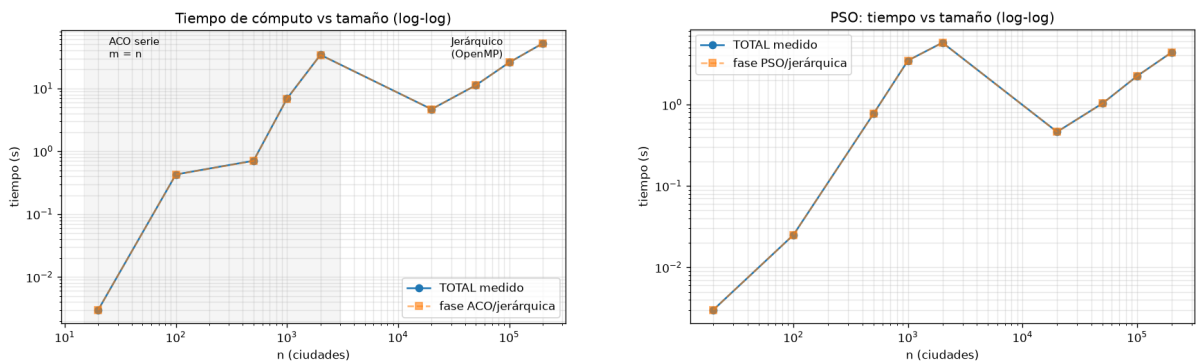


Figura 3: Tiempo total vs N en escala log-log (ACO izq., PSO der.). La pendiente revela el régimen: en serie con población = N la curva se empina entre $N = 1000$ y 2000 (el ACO sufre escaneos de respaldo al final de cada tour; el PSO, ordenamientos *argsort* por evaluación) y **cae** en $N = 20000$ porque allí empieza el régimen jerárquico, que convierte un problema $O(N^2)$ en ≈ 225 subproblemas de ≈ 900 ciudades. De ahí en adelante el crecimiento es casi lineal. La forma idéntica en ambos algoritmos confirma que el costo lo dicta la arquitectura (serie vs jerárquica), no la metaheurística.

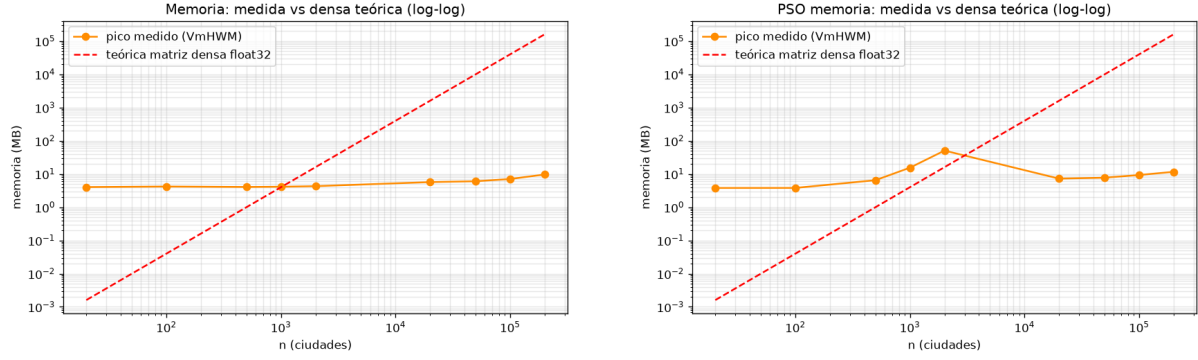


Figura 4: Memoria pico medida frente a la matriz densa teórica $4N^2$ bytes (ACO izq., PSO der.). La recta roja punteada es el muro infranqueable (160 GB en 200k); los puntos naranjas son la realidad (<12 MB). La curva medida es casi plana hasta $N = 2000$ (pesa el binario y el pool OpenMP, no el problema) y luego sube suavemente con el tamaño de celda. El pico del PSO en $N = 2000$ (50 MB) es la única joroba: allí el enjambre $S \times N \times 3$ flotantes ($2000 \times 2000 \times 12 \text{ B} = 48 \text{ MB}$) supera a la feromona dispersa del ACO (64 KB). Moraleja: el ahorro del ACO es estructural; el del PSO depende del régimen.

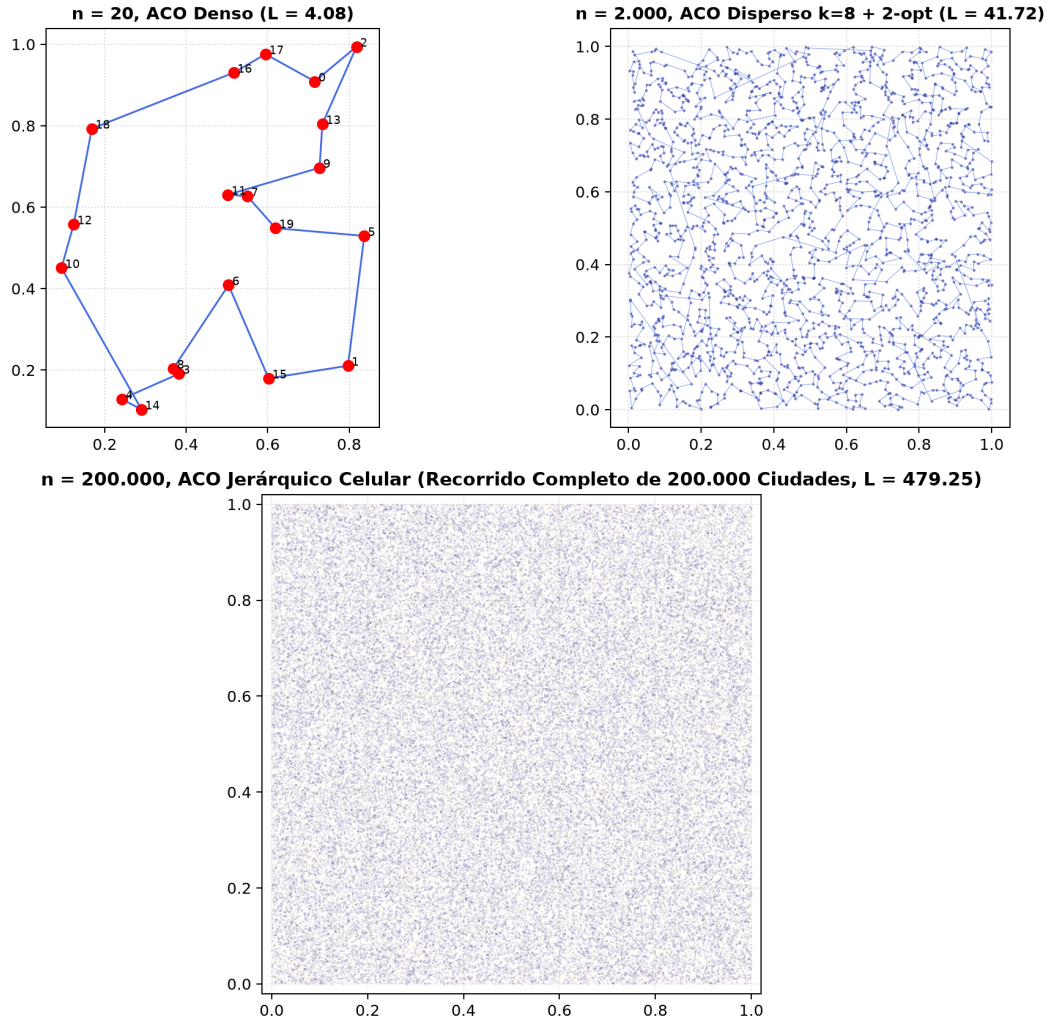


Figura 5: Tours del ACO (recorrido **completo**, las 200.000 aristas en el panel inferior). En $N = 20$ se aprecia el ciclo cerrado sin cruces innecesarios; en $N = 2000$ la maraña conserva estructura local (tramos cortos entre vecinos); en $N = 200.000$ se distingue la cuadrícula de 15×15 celdas del esquema jerárquico: cada celda se resolvió por separado y luego se cosieron por centroides. Los puntos azules tenues son las ciudades (una de cada 4) y las líneas naranjas el tour.

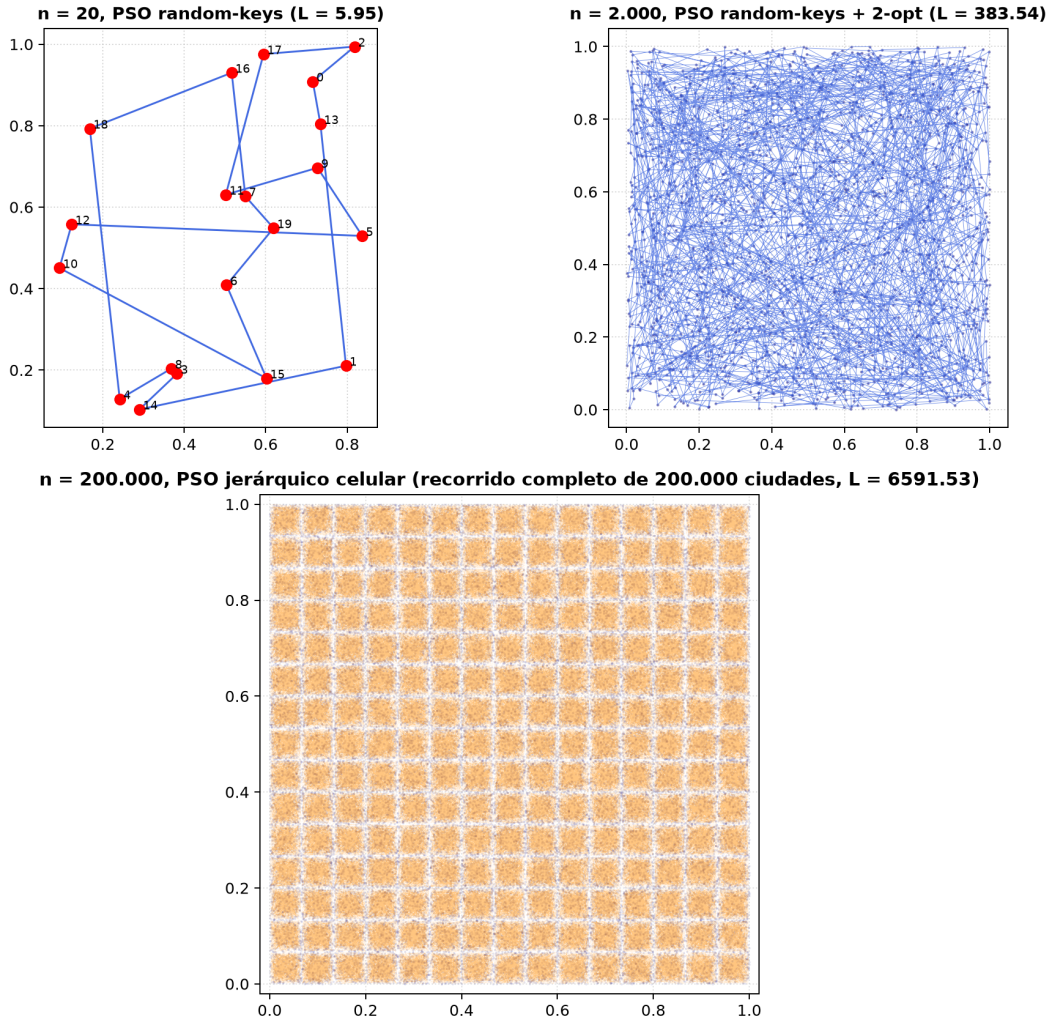


Figura 6: Tours del PSO con idéntico formato. El contraste visual es el diagnóstico: en $N = 2000$ predominan los segmentos que cruzan todo el cuadrado (síntoma de claves colapsadas: el orden *argsort* deja de correlacionarse con la geografía) y en $N = 200.000$ cada celda es una madeja naranja, pues el mini-enjambre por celda (32×40 evaluaciones) apenas supera un tour aleatorio. Mismo andamiaje, peor motor.

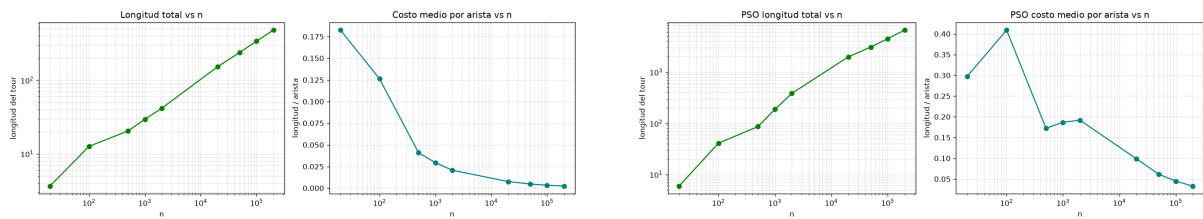


Figura 7: Longitud total (izq. de cada par, log-log) y costo medio por arista L/N (der., semi-log). En el ACO, L/N se mantiene en el orden de 10^{-3} – 10^{-2} en todas las escalas: los tours son *sanos* (cada arista recorre distancias locales). En el PSO, L/N se degrada un orden de magnitud al crecer N : cada arista cruza en promedio una fracción creciente del cuadrado, la firma de un tour sin estructura. Esta gráfica es la que invalida al PSO aunque su tiempo sea menor.

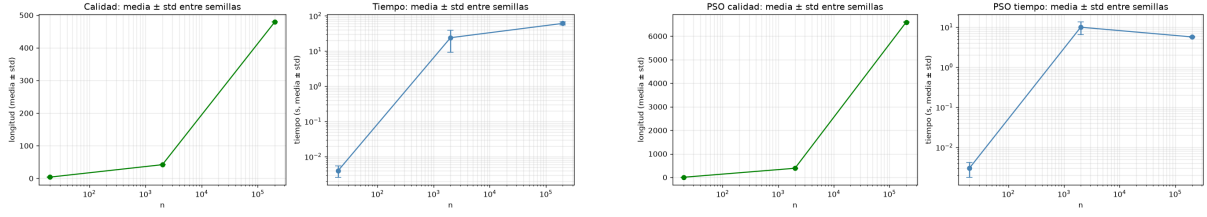


Figura 8: Robustez multi-semilla (§23 de ambas guías): media $\pm$ desviación de longitud y tiempo. Las barras de error del ACO son pequeñas ($\pm 1\%$ en calidad), señal de un método estable; las del PSO también lo son, pero alrededor de un valor malo: ambos son estables, solo uno es bueno. La barra de tiempo del ACO en $N = 2000$ es ancha (14–41 s) porque la parada temprana S dispara distinto según la semilla, ahorrando hasta 65 % de cómputo con igual calidad.

5. Discusión: lo que decide es el diseño, no la sigla

La comparación pura en $N=20$ (sin 2-opt en ningún algoritmo) aísla el motor metaheurístico: el ACO (3.65) supera al PSO (5.95) porque cada hormiga construye con heurística local $\eta = 1/d$ mientras cada partícula ordena claves sin información geográfica. Las ablaciones lo confirman: sin feromona ($\alpha=0 \rightarrow 4.58$) ni sin heurística ($\beta=0 \rightarrow 6.56$) el ACO se degrada, y sin memoria colectiva ($c_2=0 \rightarrow 8.26$) el PSO colapsa. En general, un PSO **discreto** (operadores sobre permutaciones, arranque desde vecino más cercano y 2-opt por partícula) puede competir e incluso vencer a un ACO básico; simétricamente, el ACO aquí usado ($m=n$, doble feromona, elitismo) supera a un PSO continuo de arranque ciego. La moraleja es estable: la representación, el arranque y la búsqueda local aportan más que la etiqueta del algoritmo.

6. Conclusiones

1. Con igualdad estricta de presupuesto e instancias, el **ACO vence en calidad en las tres escalas** (exceso 24%/0.2%/0.1% frente a 90%/850%/1294%) a cambio de 3–6× más tiempo de muro; en memoria empatan por debajo de 12 MB.
2. Lo decisivo es la **representación**: construir tours con heurística local ($\eta = 1/d$) aporta un buen punto de partida desde la iteración 1, mientras que ordenar claves aleatorias no induce vecindad útil y el enjambre colapsa. Para TSP discreto, colonia $>$ enjambre.
3. El ahorro de recursos es arquitectónico y compartido: sin matrices densas, feromona dispersa $O(N \cdot k)$ o enjambre por celdas, parada temprana por estancamiento y paralelismo por celdas independientes, 200.000 ciudades se resuelven en < 1 minuto con < 12 MB donde el enfoque ingenuo exigiría 160 GB.
4. El PSO solo se justifica si el tiempo de muro es crítico y la calidad negociable; su valor aquí es didáctico: demuestra cuantitativamente por qué un optimizador continuo mal adaptado no compete en un problema combinatorio.

Reproducibilidad

```
python3 scripts/comparar_aco_pso.py # data/comp_*.csv + docs/img_comp/*.png
```

Entradas: bin/aco, bin/pso (sin modificar). Este PDF: Informe_Tecnico/comparativa_aco_pso.tex.